

Week 13 Exercise Handout

McPAT-Based Energy Optimization with gem5

Open Source Prototype Systems and Applications

1. Exercise goal and learning outcome

- **Goal:** Use gem5 statistics and McPAT estimates to compare controlled energy-oriented design choices.
- **Main idea:** Run the same RISC-V workload under a baseline and one or more variants, then compute runtime, average power, total energy, EPI, and EDP.
- **Reasoning target:** Explain whether the energy change mainly comes from runtime, activity, leakage, or memory/cache behavior.
- **Deliverable:** Submit one results table, one short explanation, and one design recommendation.

2. Required environment

- **Docker image:** Use the prepared image that contains gem5, McPAT, Verilator, RISC-V compiler tools, and Python analysis packages.

```
docker pull amansinhaatnycu/osp:week13-power
```

```
docker run --rm -it -v "$(pwd)"/work -w /work amansinhaatnycu/osp:week13-power bash
```

- **Mac and non-Mac use:** The same command should select the correct image platform automatically. Apple Silicon users may add --platform linux/arm64 if needed; x86 Linux users may add --platform linux/amd64 if needed.
- **Tool check:** Inside Docker, run the provided checking script before starting the exercise.

```
cd /work/week13_mcpat_exercise_src  
bash scripts/00_check_tools.sh
```

3. Files in the exercise package

Path	Purpose
workloads/array_sum.c	Default deterministic memory-access workload
workloads/branchy_loop.c	Branch-heavy alternative workload
workloads/mini_matmul.c	Small compute/locality workload
scripts/01_build_workloads.sh	Builds a static RISC-V binary
scripts/02_run_gem5_sweep.sh	Runs baseline, cache variant, and frequency variant
scripts/03_generate_mcpat_xml.py	Creates McPAT XML from gem5 stats and a McPAT template
scripts/04_run_mcpat_sweep.sh	Runs McPAT for completed gem5 runs
scripts/05_summarize_results.py	Computes energy, EPI, and EDP into summary files
templates/student_results_table.csv	Blank table for recording results

4. Recommended one-hour schedule

Time	Activity	Output
0-10 min	Start Docker, check tools, inspect workload and scripts	Working environment
10-25 min	Build workload and run baseline gem5 simulation	Baseline stats.txt and config files
25-40 min	Run one or two controlled variants	Variant stats.txt and run metadata
40-50 min	Generate McPAT XML, run McPAT, summarize metrics	summary.csv and summary.md
50-60 min	Interpret mechanism and choose best configuration	Table, explanation, recommendation

5. Manual workflow and commands

- **Build step:** Compile the selected C workload into a static RISC-V binary.

```
bash scripts/01_build_workloads.sh workloads/array_sum.c
```

- **gem5 step:** Run the baseline and controlled variants. Each run stores stats, configs, terminal output, and run metadata under results/.

```
bash scripts/02_run_gem5_sweep.sh build/array_sum.riscv
```

- **McPAT step:** Convert each gem5 run into McPAT XML and run McPAT.

```
bash scripts/04_run_mcpat_sweep.sh
```

- **Summary step:** Compute normalized metrics for comparison.

```
python3 scripts/05_summarize_results.py results
```

6. Controlled design variants

Configuration	Changed variable	Expected benefit	Expected cost
baseline	None	Reference behavior	None
l1d_big	L1 data cache size/associativity	Lower miss rate and reduced lower-level traffic	Higher local cache access energy and possible leakage increase
freq_fast	Core clock assumption	Shorter simulated runtime	Higher average power and possible limited benefit if memory-bound

- **Control rule:** Change only one major design variable per experiment whenever possible.
- **Fairness rule:** Use the same binary, input size, compiler options, and measured interval for all configurations.

7. Required metrics

Metric	Meaning	How to compute
Runtime T	Execution interval in seconds	Read simSeconds from gem5, or convert ticks/cycles if needed
Average power P _{avg}	Dynamic plus leakage power estimate	Extract from McPAT report
Total energy E	Total joules for the measured interval	$E = P_{avg} \times T$
EPI	Energy per committed instruction	$EPI = E / \text{committed instructions}$
EDP	Energy-delay product	$EDP = E \times T$
Miss behavior	Memory hierarchy activity signal	Use D-cache miss/access counters from gem5

8. Interpretation guide

- **Runtime effect:** Did the variant reduce simulated seconds or cycles enough to matter?
- **Activity effect:** Did miss rate, memory traffic, or dynamic power change in the expected direction?
- **Leakage effect:** Did shorter runtime reduce leakage energy, or did larger structures increase leakage power?
- **Energy-vs-performance trade-off:** A lower runtime is not automatically better if power rises enough to increase energy or EDP.
- **Cache-specific reasoning:** A larger cache helps only when avoided misses outweigh higher local array energy and leakage.
- **Frequency/core reasoning:** A faster core helps compute-bound work more than memory-bound work; memory stalls can waste extra dynamic power.

9. Common mistakes to avoid

- **Watts-only claim:** Lower power is not enough; report total energy and runtime.
- **Joules-only claim:** Total energy must be tied to workload size or useful work.
- **Counter mismatch:** Do not combine gem5 stats from one run with McPAT XML from another run.
- **Warm-up confusion:** Avoid mixing initialization behavior with the intended kernel interval unless explicitly stated.
- **Overgeneralization:** Do not claim one cache size, core type, or frequency is best for all workloads.
- **Unverified conversion:** Check the McPAT XML patch report and record which template was used.

10. Student submission format

- **Explanation:** Write 5-8 sentences explaining the main source of energy change.
- **Recommendation:** Choose the preferred configuration and justify it using energy, EPI, and EDP.
- **Limitations:** State that McPAT is an architectural model and not post-layout power signoff.

Config	T (s)	Pavg (W)	E (J)	EPI (J/inst)	EDP (J-s)	Main explanation
Baseline						Reference
l1d_big						Cache/memory effect
freq_fast						Runtime/power effect